

UNIVERZA ALMA MATER EUROPAEA

Fakulteta za informacijske tehnologije in računalništvo

Informacijski Sistemi — 3. letnik

Sistem za zaščito pred izgubo podatkov

Izolirana arhitektura varnostnega kopiranja v oblaku AWS

Projektna dokumentacija

Avtor: Nejc Dolinar

Predmet: Računalništvo v oblaku

Ljubljana, april 2026

POVZETEK

Dokumentacija opisuje zasnovo in izvedbo sistema za varnostno kopiranje podatkov v okolju Amazon Web Services. Sistem temelji na principu zaščite pred izgubo podatkov (angl. Data Loss Prevention) in uporablja dve ločeni instanci EC2: javno dostopno vstopno vozlišče (Ingress) in izolirano arhivsko vozlišče (Vault). Med vozliščema teče samo šifriran tunel WireGuard.

Za nalaganje datotek uporabniki uporabljajo Nextcloud, varnostne kopije pa nastajajo samodejno z orodjem Restic. Vsaka kopija se kriptografsko preveri, vmesne datoteke se po uspešni replikaciji varno izbrišejo. Sistem sem preizkusil s popolno obnovo podatkov: zgoščene vrednosti SHA-256 pred izbrisom in po obnovi se ujemajo, kar pomeni, da so podatki obnovljivi do bita natančno.

Ključne besede: AWS, varnostno kopiranje, DLP, WireGuard, Restic, Nextcloud, izolacija, šifriranje, Docker.

KAZALO VSEBINE

1	Uvod	5
2	VPC – omrežna infrastruktura	5
3	Podomrežja (Subnets)	6
3.1	Usmerjevalna tabela javnega podomrežja	6
3.2	Usmerjevalna tabela zasebnega podomrežja	7
4	Skupine varnosti	7
4.1	ingress-sg (skupina za Ingress)	7
4.2	vault-sg (skupina za Vault)	8
5	SSH ključni par	9
6	Izgradnja EC2 inštanc	9
6.1	Vstopno vozlišče (Ingress)	9
6.2	Arhivsko vozlišče (Vault)	11
7	Namestitev Docker-ja na izolirano vozlišče	13
7.1	Prenos Docker GPG ključa	13
7.2	Namestitev preko apt-cacher-ng proxy-ja	13
8	Namestitev WireGuard – offline (air-gapped)	14
9	WireGuard – šifriran tunel	15
9.1	Generiranje ključnih parov	15
9.2	Konfiguracija na strani Vault	16
9.3	Konfiguracija na strani Ingress	16
10	Vzpostavitev tunela	17
10.1	Preizkus delovanja	17
11	Nextcloud – aplikacijski sloj	18
11.1	Priprava strukture	18
11.2	Docker Compose	18
11.3	Spletni vmesnik in testne datoteke	19
12	Restic REST strežnik na Vault	20
12.1	Prenos Docker slike na izolirano vozlišče	21
12.2	HTTP osnovna avtentikacija	21
12.3	Docker Compose za Restic	21
12.4	Inicializacija Restic repozitorija	22
13	Funkcija »preveri-in-uniči«	23

14	Test obnove podatkov (DR test).....	24
15	Zaključek	24
15.1	Povzetek doseženega	24
15.2	Znane omejitve	25
15.3	Možnosti za nadaljnji razvoj.....	25
16	Viri in literature	26

1 Uvod

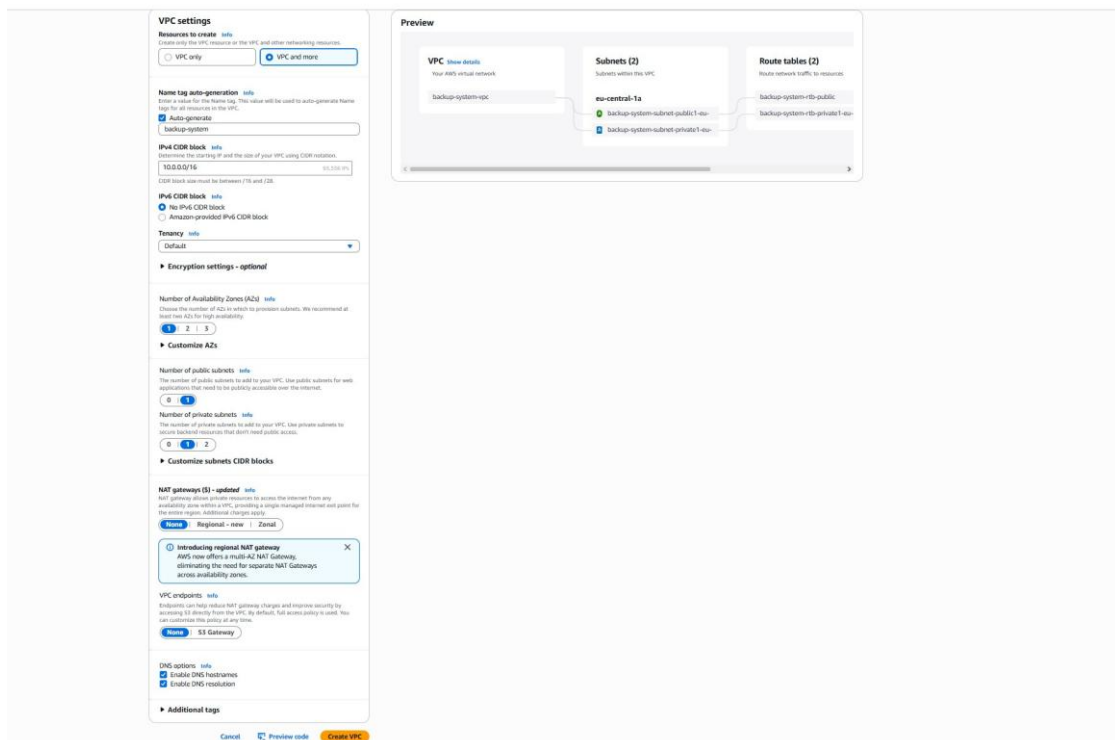
Izguba podatkov je resna nevarnost. Napadi z izsiljevalsko programsko opremo, strojne okvare in človeške napake lahko hitro uničijo digitalne podatke organizacije ali posameznika. V branži se je za zaščito pred temi tveganji uveljavilo pravilo 3-2-1: tri kopije podatkov na dveh različnih medijih, ena od kopij izven lokacije.

V projektni nalogi sem zasnoval in postavil sistem za varnostno kopiranje, ki uporablja izolacijo, šifriranje med prenosom in v mirovanju, večnivojsko avtentikacijo in samodejno preverjanje celovitosti. Vse je postavljeno na brezplačnem tarifnem paketu AWS in odprtokodni programski opremi.

Arhitektura uporablja dva ločena virtualna strežnika v oblaku AWS: vstopno vozlišče (Ingress) je izpostavljeno internetu in gostuje aplikacijo Nextcloud, arhivsko vozlišče (Vault) je izolirano in shranjuje šifrirane varnostne kopije. Med njima je samo en kanal komunikacije: šifriran tunel WireGuard.

2 VPC – omrežna infrastruktura

Z uporabo AWS čarovnika »VPC and more« sem ustvaril virtualno zasebno omrežje (Virtual Private Cloud) z naslovnim prostorom 10.0.0.0/16 v regiji eu-central-1 (Frankfurt). Regija je bila izbrana zaradi nizke latence iz Slovenije in skladnosti z evropsko zakonodajo o varstvu podatkov (GDPR). VPC vsebuje eno javno podomrežje z internetnim preходом in eno zasebno podomrežje brez NAT prehoda.



Slika 1: Konfiguracija VPC v AWS čarovniku — 10.0.0.0/16, ena cona dostopnosti, NAT prehod nastavljen na »None«

NAT prehoda namerno nisem postavil. NAT bi sicer omogočil arhivskemu vozlišču izhodni promet (na primer za samodejne posodobitve operacijskega sistema), a bi bil istočasno tudi pot za napadalca. Brez NAT te poti preprosto ni: zlonamerno programje, ki bi se kakorkoli znašlo na vozlišču, ne bi imelo kam poslati podatkov.

3 Podomrežja (Subnets)

Znotraj VPC sta bili ustvarjeni dve podomrežji — vsako z 4091 razpoložljivimi IPv4 naslovi. Javno podomrežje »ingress-public« (10.0.0.0/20) je namenjeno vstopnemu vozlišču, ki potrebuje stik z internetom. Zasebno podomrežje »vault-private« (10.0.128.0/20) gosti arhivsko vozlišče in ne sme imeti nobene poti navzven.

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR association ID	Available IPv4 addresses	Availability Zone
ingress-public	subnet-09482ba9efc24846d	Available	vpc-0b9b845604ba8a6dc bac...	OFF	10.0.0.0/20	-	-	4091	eu-central-1a
vault-private	subnet-0263c0f1295aed9af	Available	vpc-0b9b845604ba8a6dc bac...	OFF	10.0.128.0/20	-	-	4091	eu-central-1a

Slika 2: Pregled obeh podomrežij v AWS konzoli

3.1 Usmerjevalna tabela javnega podomrežja

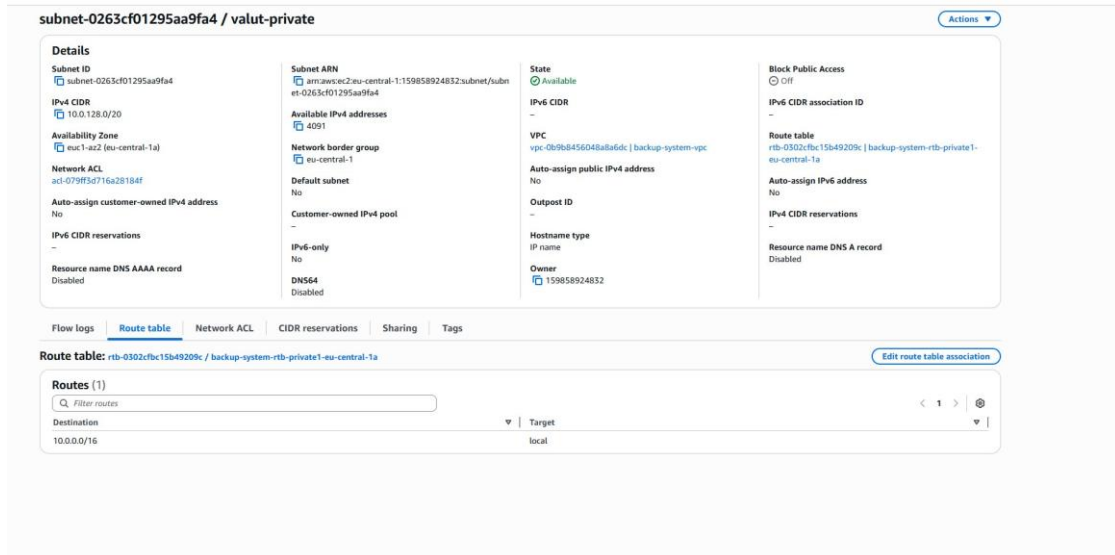
Javno podomrežje ima dve poti: lokalno (10.0.0.0/16) za komunikacijo znotraj VPC in privzeto (0.0.0.0/0), ki kaže na internetni prehod (Internet Gateway, igw-0815c067f914989d4). To pomeni, da lahko Ingress doseže katerokoli storitev v internetu in obratno — internet lahko doseže Ingress preko njegovega javnega IP naslova.

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-0815c067f914989d4

Slika 3: Usmerjevalna tabela javnega podomrežja — privzeta pot 0.0.0.0/0 vodi v internetni prehod

3.2 Usmerjevalna tabela zasebnega podomrežja

Zasebno podomrežje ima samo eno pot, lokalno (10.0.0.0/16). Privzete poti (0.0.0.0/0) ni, zato Vault ne more poslati paketa v internet. To omejitev AWS uveljavlja na ravni hipervizorja in je ni mogoče zaobiti niti z root dostopom v gostujočem operacijskem sistemu.



Slika 4: Usmerjevalna tabela zasebnega podomrežja — vsebuje samo lokalno pot, brez kakršnekoli internetne povezljivosti

4 Skupine varnosti

Skupine varnosti (Security Groups) v AWS predstavljajo virtualne požarne zidove na ravni omrežnega vmesnika vsake EC2 inštanca. V okviru tega projekta sta bili ustvarjeni dve: ingress-sg za vstopno vozlišče in vault-sg za arhivsko vozlišče. Vsaka ima striktno definirana vhodna pravila po principu najmanjših privilegijev.

4.1 ingress-sg (skupina za Ingress)

Vstopno vozlišče sprejema štiri vrste vhodnega prometa. SSH (TCP 22) je dovoljen samo iz mojega domačega IP naslova (84.52.159.184/32), kar prepreči poskuse vdora od kogarkoli drugega. HTTP (TCP 80) in HTTPS (TCP 443) sta dovoljena iz interneta zaradi delovanja Nextcloud aplikacije. WireGuard (UDP 51820) je odprt iz interneta z opombo, da bo kasneje zaklenjen na specifičnega peera. Izhodni promet je dovoljen v celoti, kar je nujno za apt posodobitve, Docker prenose in Let's Encrypt certifikate.

Create security group [Info](#)
A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)
ingress-sg
Name cannot be edited after creation.

Description [Info](#)
Public-facing Nextcloud + Duplicati host

VPC [Info](#)
vpc-0b9b8456048a8afdc (backup-system-vpc)

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Description - optional	
SSH	TCP	22	My IP	Admin access from home	Delete
HTTP	TCP	80	Anywh...	Caddy ACME challenge	Delete
HTTPS	TCP	443	Anywh...	Nextcloud web UI	Delete
Custom UDP	UDP	51820	Anywh...	WireGuard (locked further later)	Delete

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Outbound rules [Info](#)

Type	Protocol	Port range	Destination	Description - optional	
All traffic	All	All	Custom		Delete

[Add rule](#)

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

Slika 5: Skupina varnosti »ingress-sg« — SSH samo iz mojega IP, HTTP/HTTPS javno, WireGuard UDP 51820

4.2 vault-sg (skupina za Vault)

Arhivsko vozlišče ima še strožja pravila. SSH (TCP 22) in WireGuard (UDP 51820) sta dovoljena izključno iz vira, ki je sama varnostna skupina ingress-sg (sg-0c5b030a301019e47). To pomeni, da lahko Vault doseže samo inštanca, ki ima ingress-sg skupino — torej Ingress. Izhodnih pravil ni, kar dodatno preprečuje morebiten outbound promet z Vault-a.

Create security group [Info](#)
A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)
vault-sg
Name cannot be edited after creation.

Description [Info](#)
Isolated backup vault, accepts traffic only from Ingress

VPC [Info](#)
vpc-0b9b8456048a8afdc (backup-system-vpc)

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Description - optional	
Custom UDP	UDP	51820	Custom	WireGuard from Ingress only	Delete
SSH	TCP	22	Custom	Bastion access via Ingress only	Delete

[Add rule](#)

Outbound rules [Info](#)

This security group has no outbound rules.

[Add rule](#)

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
No tags associated with the resource.

[Add new tag](#)

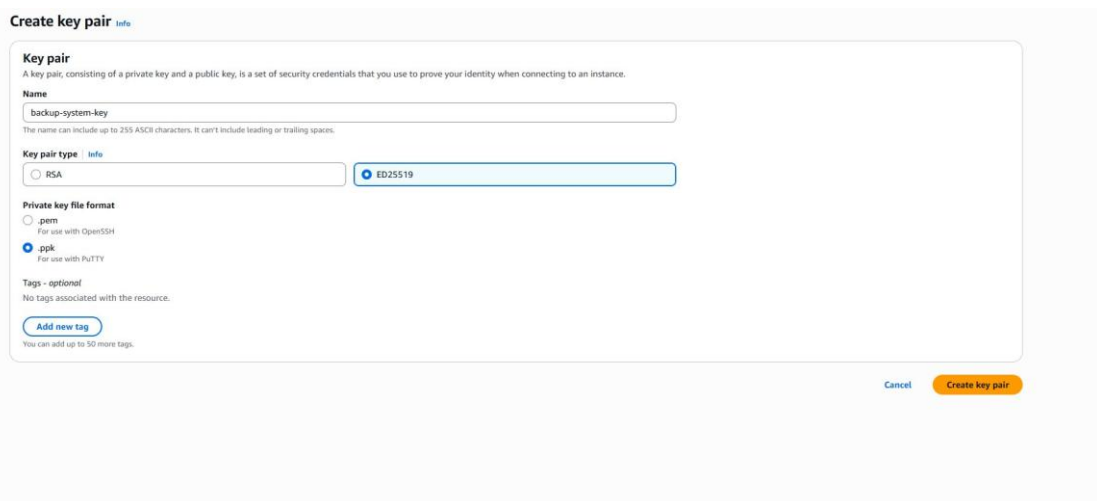
You can add up to 50 more tags.

[Cancel](#) [Create security group](#)

Slika 6: Skupina varnosti »vault-sg« — vhodni promet samo iz ingress-sg, brez izhodnih pravil

5 SSH ključni par

Pred zagonom prve EC2 inštanca sem ustvaril SSH ključni par za dostop do obeh strežnikov. Izbral sem algoritem ED25519, ki ima krajše ključe in hitrejši podpis kot RSA. Datoteko .ppk uporabim z odjemalcem PuTTY na Windowsih.



Create key pair [Info](#)

Key pair
A key pair, consisting of a private key and a public key, is a set of security credentials that you use to prove your identity when connecting to an instance.

Name
backup-system-key
The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type [Info](#)
☐ RSA ☒ ED25519

Private key file format
☐ .pem For use with OpenSSH
☒ .ppk For use with PuTTY

Tags - optional
No tags associated with the resource.
[Add new tag](#)
You can add up to 50 more tags.

[Cancel](#) [Create key pair](#)

Slika 7: Ustvarjanje SSH ključnega para »backup-system-key« v AWS konzoli

Ključ je shranjen lokalno na razvojnem računalniku. Vault je dosegljiv samo preko Ingress-a po t. i. SSH bastion vzorcu, kar pomeni, da isti ključ uporabim za hop preko Ingress-a do Vault-a. Privatni ključ se nikoli ne kopira na strežnik; Pageant ga skozi Ingress posreduje po agent forwarding-u.

6 Izgradnja EC2 inštanca

6.1 Vstopno vozlišče (Ingress)

Vstopno vozlišče poganja Ubuntu Server 24.04 LTS na inštanca tipa t3.micro (1 vCPU, 1 GiB pomnilnika), ki je v okviru AWS Free Tier paketa. Inštanca je v javnem podomrežju z dodeljenim javnim IPv4 naslovom. Velikost diska je 12 GiB EBS gp3, kar pušča dovolj prostora za operacijski sistem, Docker slike in delovne podatke Nextcloud-a.

It seems like you may be new to searching instances in EC2. Take a walkthrough to learn about EC2, how to launch instances and about best practices. [Take a walkthrough](#) [Do not show me this message again](#)

Launch an instance

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags

Name

[Add additional tags](#)

Application and OS images (Amazon Machine Image)

As AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recently Quick Start

Amazon Linux

macOS

Ubuntu

Windows

RHEL

SUSE Linux

Debian

[Browse more AMIs](#)
Includes AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-05832a37f105d545aa / ami-05832a37f105d545aa (34-bit)

Description
Ubuntu Server 24.04 LTS (HVM), SSD Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/remote>).

Canonical, Ubuntu, 24.04, amd64 native image

Architecture	AMI ID	Publication Date	Username	Original product
64-bit (x86)	ami-05832a37f105d545aa	2024-03-14	ubuntu	Canonical

Instance type

Instance type

t3.micro
Partially on-chip CPU, 1 GiB memory, Current generation Linux On-Demand 60MB new pricing \$0.012 USD per hour
On-Demand t3.micro new pricing \$0.012 USD per hour On-Demand t3.micro new pricing \$0.012 USD per hour On-Demand t3.micro new pricing \$0.012 USD per hour

Free tier eligible All generations [Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

Key pair (login)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name *required*

Backup-system-key

[Create new key pair](#)

Network settings

VPC *required*

ap-05832a37f105d545aa (Backup-system-key)

Subnet

subnet-04623a7f248452
VPC: ap-05832a37f105d545aa Subnet: 10.0.0.0/24 Availability Zone: us-east-1a (us-east-1a) Ingress: public

[Create new subnet](#)

Auto-assign public IP

Enable

Placement (security group)

[Create security group](#) [Select existing security group](#)

Common security group

ingress-ig-0c5053a01019a47

[Compare security group rules](#)

Security groups that you add or remove here will be added to or removed from all your network interfaces.

Advanced network configuration

Configure storage

Size: 12 GiB gp3 Root volume, 5000 IOPS, Not encrypted

[Add new volume](#)

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance.

Click which to view backup information
This tag that you assign determines whether the instance will be backed up by any Data Lifecycle Manager policies.

File systems

☐ EFS ☐ FSx ☐ S3 File - new

Summary

Number of instances: 1

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64, root-meag
ami-05832a37f105d545aa

Virtual server type (instance type)
t3.micro

Placement (security group)
ingress-ig

Storage (instances)
1 volume(s) 12 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Slika 8: Konfiguracija Ingress inštanca — Ubuntu 24.04, t3.micro, omogočen javni IP

Pri zagonu inštanca sem v polje »User data« vnesel bash skripto, ki samodejno izvede začetno namestitvev. Skripta posodobi sistem, namesti potrebne pakete (curl, gnupg, fail2ban, htop), doda Docker repozitorij in namesti Docker engine ter Compose vtičnik. Posebej določi tudi hostname inštanca na »ingress«, kar pomaga pri orientaciji v terminalu.

User data - optional

Upload a file with your user data or enter it in the field.

[Choose file](#)

```

apt-get update -y
apt-get upgrade -y
apt-get install -y ca-certificates curl gnupg lsb-release ufw fail2ban htop
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" >
/etc/apt/sources.list.d/docker.list
apt-get update -y
apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
usermod -aG docker ubuntu
systemctl enable --now docker
hostnamectl set-hostname ingress
  
```

☐ User data has already been base64 encoded

Slika 9: User data skripta za samodejno namestitev Docker-ja in osnovnih orodij ob prvem zagonu

```

ubuntu@ingress:~$ sudo cloud-init status --wait
status: done
ubuntu@ingress:~$ docker --version
Docker version 29.4.1, build 055a478
ubuntu@ingress:~$ docker compose version
Docker Compose version v5.1.3
ubuntu@ingress:~$ hostname
ingress
ubuntu@ingress:~$

```

Slika 10: Verifikacija namestitve na Ingress — cloud-init končan, Docker 29.4.1 in Compose v5.1.3 delujeta, hostname potrjen

6.2 Arhivsko vozlišče (Vault)

Arhivsko vozlišče prav tako poganja Ubuntu 24.04 na t3.micro inštanci, vendar z drugačnimi nastavitvami. Disk EBS ima 16 GiB (več prostora za varnostne kopije), javni IPv4 naslov pa je onemogočen že ob ustvarjanju inštanca. Te nastavitve ni mogoče spremeniti naknadno: na to inštanco se javni IP ne more dodeliti, dokler obstaja.

The screenshot displays the AWS Management Console configuration page for a new EC2 instance. The instance is named 'vault' and is being configured with the following settings:

- Application and OS Images (Amazon Machine Image):** Ubuntu Server 24.04 LTS (HVM), SSD Volume Type. The AMI ID is ami-05852c3f795d545ea, published on 2024-05-14. The architecture is 64-bit (x86).
- Instance type:** t3.micro. The instance is configured with 2 vCPUs and 1 GB of memory. The pricing is set to 'Free tier eligible'.
- Key pair (login):** backup-system-key. The key pair is required for the instance to be launched.
- Network settings:**
 - VPC:** vpc-0b90845048a8a6d6 (backup-system-vpc).
 - Subnet:** subnet-0263c01295aa39a4. The subnet is configured with a VPC ID of vpc-0b90845048a8a6d6, a CIDR of 10.0.0.0/16, and a VPC type of IPv4-only.
 - Auto-assign public IP:** Disable.
 - Firewall (security group):** Select existing security group. The security group is vault-sg-sp-0c12e518005ae4d02.

The right-hand side of the console shows a summary of the configuration, including the number of instances (1), the software image (AMI), the virtual server type (instance type), the firewall (security group), and the storage (volumes).

Slika 11: Konfiguracija Vault instance — Auto-assign public IP nastavljen na »Disable«, izbrano podomrežje vault-private

Storage (volumes) [Info](#) [Simple](#)

EBS Volumes [Hide details](#)

▼ Volume 1 (AMI Root) (Custom)

Storage type Info EBS	Device name - required Info /dev/sda1	Snapshot Info snap-07da3848dcb588589
Size (GiB) Info 16	Volume type Info gp3 Free tier eligible ▼	IOPS Info 3000
Delete on termination Info Yes ▼	Encrypted Info Encrypted ▼	KMS key Info (default) aws/ebs Key ID: alias/aws/ebs
Throughput Info 125	Volume initialization rate - new, optional Info Enter a value Min: 100 MiB/s, Max: 300 MiB/s. Additional charges apply L2	

[Add new volume](#)

The selected AMI contains instance store volumes, however the instance does not allow any instance store volumes. None of the instance store volumes from the AMI will be accessible from the instance

Virtual se
t3.micro

 Firewall (f
vault-sg

 Storage (f
1 volume(

 Cancel

Slika 12: Konfiguracija EBS diska na Vault — vključeno šifriranje z AWS upravljanim ključem (aws/ebs)

Šifriranje diska EBS poskrbi, da so podatki na disku zavarovani tudi v primeru, da bi nekdo fizično pridobil dostop do AWS strežnikov. Zaščita deluje na ravni AWS infrastrukture in je za operacijski sistem transparentna. Skupaj z Restic šifriranjem na ravni datotek (več o tem kasneje) so podatki v mirovanju zaščiteni z dvema ločenima sloja.

```
Last login: Sun Apr 26 19:03:43 2026 from 84.52.159.184
ubuntu@ingress:~$ ssh ubuntu@10.0.129.64
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun Apr 26 19:06:24 UTC 2026

System load:  0.0          Temperature:   -273.1 C
Usage of /:   12.4% of 14.46GB  Processes:    111
Memory usage: 26%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 10.0.129.64

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@vault:~$ hostname
vault
ubuntu@vault:~$ ip -4 addr show | grep inet
    inet 127.0.0.1/8 scope host lo
    inet 10.0.129.64/20 metric 100 bnd 10.0.143.255 scope global dynamic ens5
ubuntu@vault:~$ curl -m 5 https://google.com
curl: (28) Connection timed out after 5001 milliseconds
ubuntu@vault:~$ ^C
ubuntu@vault:~$
```

Slika 13: Dokaz izolacije — `hostname` vrne »vault«, `ip addr` prikazuje samo zasebni 10.0.x naslov, `curl https://google.com` pa se po 5 sekundah časovno prekorači

Časovna prekoračitev `curl` ukaza dokazuje, da Vault dejansko nima poti do interneta. To je pomembno z varnostnega vidika: če Vault ne more navzven, potem tudi morebitno zlonamerno programje na njem ne more komunicirati z napadalčevo infrastrukturo.

7 Namestitev Docker-ja na izolirano vozlišče

Težava izoliranih sistemov: kako namestiti programsko opremo, če stroj nima dostopa do interneta? V tem projektu Ingress, ki ima internet, deluje kot začasni proxy oz. distribucijska točka za Vault — t. i. air-gapped vzorec.

7.1 Prenos Docker GPG ključa

Najprej sem na Ingress-u prenesel GPG ključ, s katerim Docker podpisuje svoje pakete. Ta ključ se uporablja za preverjanje verodostojnosti vsakega paketa, ki ga `apt` prenese. Z ukazom `curl` sem ga shranil v `/tmp/docker.gpg` in nato z `scp` prenesel na Vault preko zasebnega omrežja VPC.

```
last login: Sun Apr 26 19:27:35 2026 from 84.52.159.184
buntu@ingress:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /tmp/docker.gpg
buntu@ingress:~$ ls -lh /tmp/docker.gpg
-rw-rw-r-- 1 ubuntu ubuntu 3.8K Apr 26 19:29 /tmp/docker.gpg
buntu@ingress:~$ scp /tmp/docker.gpg ubuntu@10.0.129.64:/tmp/
docker.gpg
buntu@ingress:~$
```

Slika 14: Prenos Docker GPG ključa — `curl` na Ingress (kjer je internet), nato `scp` preko zasebnega omrežja na Vault

```
last login: Sun Apr 26 19:06:25 2026 from 10.0.8.58
buntu@vault:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /tmp/docker.gpg
buntu@vault:~$ ls -lh /tmp/docker.gpg
-rw-rw-r-- 1 ubuntu ubuntu 3.8K Apr 26 19:29 /tmp/docker.gpg
buntu@vault:~$
```

Slika 15: Prejem Docker GPG ključa na Vault — datoteka je sedaj na voljo lokalno za nadaljnjo uporabo z `apt-jem`

7.2 Namestitev preko `apt-cacher-ng` proxy-ja

Za sam `apt` promet sem na Ingress-u postavil začasen `apt-cacher-ng` proxy strežnik. Konfiguracijska datoteka `/etc/apt/apt.conf.d/00proxy` na Vault-u je preusmerila vse `apt` zahteve preko Ingress-a. Ko je bila začetna programska oprema (Docker engine, Docker Compose, runtime knjižnice) nameščena, sem proxy vrata 3142 v skupinah varnosti zaprl. Vault se je tako vrnil v popolnoma izolirano stanje, vendar z že nameščenim Docker-jem.

8 Namestitev WireGuard – offline (air-gapped)

Po namestitvi Docker-ja sem proxy zaprl in se odločil, da bom WireGuard namestil brez interneta — s tem dokažem, da arhitektura zdrži tudi pravo izolacijo. Postopek: .deb pakete prenesem na Ingress (kjer je internet), z scp jih preko zasebnega omrežja prestavim na Vault in tam namestim z dpkg.

```
ubuntu@ingress:~$ sudo apt-get install -y Wireguard
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  wireguard-tools
The following NEW packages will be installed:
  wireguard wireguard-tools
0 upgraded, 2 newly installed, 0 to remove and 3 not upgraded.
Need to get 92.2 kB of archives.
After this operation, 345 kB of additional disk space will be used.
Get:1 http://eu-central-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 wireguard-tools amd64 1.0.20210914-lubuntu4 [89.1 kB]
Get:2 http://eu-central-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 wireguard all 1.0.20210914-lubuntu4 [3086 B]
Fetched 92.2 kB in 0s (4134 kB/s)
Selecting previously unselected package wireguard-tools.
(Reading database ... 73402 files and directories currently installed.)
Preparing to unpack .../wireguard-tools_1.0.20210914-lubuntu4_amd64.deb ...
Unpacking wireguard-tools (1.0.20210914-lubuntu4) ...
Selecting previously unselected package wireguard.
Preparing to unpack .../wireguard_1.0.20210914-lubuntu4_all.deb ...
Unpacking wireguard (1.0.20210914-lubuntu4) ...
Setting up wireguard-tools (1.0.20210914-lubuntu4) ...
wg-quick.target is a disabled or a static unit, not starting it.
Setting up wireguard (1.0.20210914-lubuntu4) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ingress:~$ wg --version
Invalid subcommand: '--version'
Usage: wg <cmd> [<args>]

Available subcommands:
  show: Shows the current configuration and device information
  showconf: Shows the current configuration of a given WireGuard interface, for use with 'setconf'
  set: Change the current configuration, add peers, remove peers, or change peers
  setconf: Applies a configuration file to a WireGuard interface
  addconf: Appends a configuration file to a WireGuard interface
  syncof: Synchronizes a configuration file to a WireGuard interface
  genkey: Generates a new private key and writes it to stdout
  genpsk: Generates a new preshared key and writes it to stdout
  pubkey: Reads a private key from stdin and writes a public key to stdout
You may pass '-help' to any of these subcommands to view usage.
ubuntu@ingress:~$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
ubuntu@ingress:~$
```

Slika 16: Namestitev WireGuard na Ingress preko apt-get — ta inštalacija služi tudi kot vir paketov za nadaljnji prenos

```
ubuntu@ingress:~$ mkdir -p ~/wg-debs
ubuntu@ingress:~$ cd ~/wg-debs
ubuntu@ingress:~/wg-debs$ sudo apt-get install -y --download-only --reinstall wireguard wireguard-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 2 reinstalled, 0 to remove and 3 not upgraded.
Need to get 0 B/92.2 kB of archives.
After this operation, 0 B of additional disk space will be used.
Download complete and in download only mode
ubuntu@ingress:~/wg-debs$ sudo cp /var/cache/apt/archives/wireguard*.deb ~/wg-debs/ 2>/dev/null || true
ubuntu@ingress:~/wg-debs$ sudo cp /var/cache/apt/archives/wireguard-tools*.deb ~/wg-debs/ 2>/dev/null || true
ubuntu@ingress:~/wg-debs$ ls -lh ~/wg-debs/
total 92K
-rw-r--r-- 1 root root 88K Apr 27 10:44 wireguard-tools_1.0.20210914-lubuntu4_amd64.deb
-rw-r--r-- 1 root root 3.1K Apr 27 10:44 wireguard_1.0.20210914-lubuntu4_all.deb
ubuntu@ingress:~/wg-debs$
```

Slika 17: Prenos .deb paketov za WireGuard na Ingress z apt-get download in kopiranje v ~/wg-debs za scp prenos

```

ubuntu@vault:~$ cd /tmp
ubuntu@vault:/tmp$ ls *.deb
wireguard-tools_1.0.20210914-lubuntu4_amd64.deb wireguard_1.0.20210914-lubuntu4_all.deb
ubuntu@vault:/tmp$ sudo dpkg -i wireguard-tools_*.deb wireguard_*.deb
Selecting previously unselected package wireguard-tools.
(Reading database ... 72815 files and directories currently installed.)
Preparing to unpack wireguard-tools_1.0.20210914-lubuntu4_amd64.deb ...
Unpacking wireguard-tools (1.0.20210914-lubuntu4) ...
Selecting previously unselected package wireguard.
Preparing to unpack wireguard_1.0.20210914-lubuntu4_all.deb ...
Unpacking wireguard (1.0.20210914-lubuntu4) ...
Setting up wireguard-tools (1.0.20210914-lubuntu4) ...
wg-quick.target is a disabled or a static unit, not starting it.
Setting up wireguard (1.0.20210914-lubuntu4) ...
Processing triggers for man-db (2.12.0-4build2) ...
ubuntu@vault:/tmp$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
ubuntu@vault:/tmp$ sudo modprobe wireguard
ubuntu@vault:/tmp$ lsmod | grep wireguard
wireguard                118784  0
curve25519_x86_64        36864  1 wireguard
ip6_udp_tunnel            16384  1 wireguard
udp_tunnel                32768  1 wireguard
libcurve25519_generic    49152  2 curve25519_x86_64,wireguard
ubuntu@vault:/tmp$

```

Slika 18: Air-gapped namestitev WireGuard na Vault z `dpkg -i`, sledi `modprobe wireguard` in `lsmod`, ki potrjuje, da je modul jedra naložen

Ta vzorec je v reguliranih okoljih (bančništvo, javna uprava, obramba) standarden, saj produkcijskim sistemom v takih okoljih ni dovoljen neposreden dostop do javnih repozitorijev paketov. Posodobitve se dostavljajo nadzorovano, preko interne infrastrukture. S tem postopkom v projektu sem dokazal, da arhitektura v praksi deluje tudi takrat, ko interneta sploh ni.

9 WireGuard – šifriran tunel

WireGuard je VPN protokol, ki za vzpostavitev seje uporablja Diffie-Hellmanov ključni dogovor po krivulji Curve25519, za simetrično šifriranje pa ChaCha20-Poly1305. Vsaka stran ima par javnega in privatnega ključa; konfiguracija eksplicitno določa javni ključ druge strani. Paketi, ki niso podpisani s pričakovanim privatnim ključem, se na ravni jedra tiho zavržejo brez odgovora. Pri geslovni avtentikaciji bi napadalec lahko ugibal; tukaj je strežnik po obnašanju neviden.

9.1 Generiranje ključnih parov

Na vsaki inštanci sem ustvaril par javnega in privatnega ključa z ukazom `wg genkey | tee vault.key | wg pubkey > vault.pub`. Privatni ključ je shranjen v `/etc/wireguard/` z dovoljenji 0600, da je dostopen samo root-u. Javni ključ se nato ročno prenese na drugo stran in vnese v njeno [Peer] sekcijo.

```

ubuntu@vault:/tmp$ sudo mkdir -p /etc/wireguard
ubuntu@vault:/tmp$ cd /etc/wireguard
-bash: cd: /etc/wireguard: Permission denied
ubuntu@vault:/tmp$ sudo sh -c 'umask 077; wg genkey | tee vault.key | wg pubkey > vault.pub'
ubuntu@vault:/tmp$ sudo cat vault.pub
NPUwEceQgTCVYxdca1GPlPaqcbGTeU0vjb/Jqk5UJGQ=
ubuntu@vault:/tmp$ ^C
ubuntu@vault:/tmp$

```

Slika 19: Generiranje WireGuard ključnega para na Vault

```

sudo mkdir -p /etc/wireguard
ubuntu@ingress:~$ sudo sh -c 'umask 077; cd /etc/wireguard && wg genkey | tee ingress.key | wg pubkey > ingress.pub'
ubuntu@ingress:~$ sudo cat /etc/wireguard/ingress.pub
bpYKVMPlIRLagKzqetD0EDU3JQV5y5XuL0Vgc54/2V4=
ubuntu@ingress:~$ sudo cat /etc/wireguard/ingress.key
eKv5QJSKG6+Oj+XgbfBwahqEbogaOtgof6vS+l/WoHM=
ubuntu@ingress:~$ sudo cat /etc/wireguard/vault.key
cat: /etc/wireguard/vault.key: No such file or directory
ubuntu@ingress:~$

```

Slika 20: Generiranje WireGuard ključnega para na Ingress

9.2 Konfiguracija na strani Vault

Vault je konfiguriran kot »strežnik« tunela — posluša na UDP vratih 51820 in čaka, da Ingress vzpostavi povezavo. Naslov v tunelu je 10.8.0.1/24. Sekcija [Peer] vsebuje javni ključ Ingress-a in dovoljen IP 10.8.0.2/32, kar pomeni, da se sprejmejo samo paketi z izvora 10.8.0.2.

```

ubuntu@vault: /tmp
GNU nano 7.2
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = iH2D4UKNMJ3wOIcY76kqpGDJXXSK95DfvyXFzqmRoEk=
SaveConfig = false

[Peer]
PublicKey = SiigLgsvUml5tueNmiFy3eMSk5NcfHx2bA+anpOKi34=
AllowedIPs = 10.8.0.2/32

```

Slika 21: Konfiguracijska datoteka /etc/wireguard/wg0.conf na Vault

9.3 Konfiguracija na strani Ingress

Ingress je konfiguriran kot »klijent« tunela — aktivno vzpostavi povezavo do Vault-a. Endpoint kaže na zasebni IP Vault-a (10.0.129.64) in vrata 51820. Direktiva PersistentKeepalive = 25 zagotavlja, da Ingress vsakih 25 sekund pošlje majhen paket preko tunela, kar ohranja stanje povezave skozi morebitne stateful požarne zidove med vozliščema.

```

GNU nano 7.2 /etc/wi
[Interface]
Address = 10.8.0.2/24
PrivateKey = eKv5QJSKG6+Oj+XgbfBwahqEbogaOtgof6vS+l/WoHM=
SaveConfig = false

[Peer]
PublicKey = bpYKVMPlIRLagKzqetD0EDU3JQV5y5XuL0Vgc54/2V4=
Endpoint = 10.0.129.64:51820
AllowedIPs = 10.8.0.1/32
PersistentKeepalive = 25

```

Slika 22: Konfiguracijska datoteka /etc/wireguard/wg0.conf na Ingress

10 Vzpostavitev tunela

Po nastavljenih konfiguracijah sem na obeh straneh zagnal WireGuard storitev z ukazom `wg-quick up wg0`. Ukaz `systemctl enable wg-quick@wg0` zagotavlja, da se tunel samodejno vzpostavi tudi po ponovnem zagonu inštanca. Vrstni red zagona je pomemben: najprej Vault (ki posluša), nato Ingress (ki kliče).

```
ubuntu@vault:/tmp$ sudo wg-quick up wg0
[+] ip link add wg0 type wireguard
[+] wg setconf wg0 /dev/fd/63
[+] ip -4 address add 10.8.0.1/24 dev wg0
[+] ip link set mtu 8921 up dev wg0
ubuntu@vault:/tmp$ sudo systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /usr/lib/systemd/system/wg-quick@.service.
ubuntu@vault:/tmp$ sudo wg show
interface: wg0
  public key: 6Z0zMLKalepELy+IgIScT+dqHKffGa+Atn4NqXsGxT8=
  private key: (hidden)
  listening port: 51820

peer: eKv5QJ5KSG6+Oj+xqbfBwahqEbogaOtcqof6vS+1/WoHM=
  allowed ips: 10.8.0.2/32
ubuntu@vault:/tmp$
```

Slika 23: Zagon WireGuard storitve na Vault — `wg-quick up wg0`, omogočanje samodejnega zagona, prikaz `peer` konfiguracije

```
ubuntu@ingress:~$ sudo wg-quick up wg0
[+] ip link add wg0 type wireguard
[+] wg setconf wg0 /dev/fd/63
[+] ip -4 address add 10.8.0.2/24 dev wg0
[+] ip link set mtu 8921 up dev wg0
ubuntu@ingress:~$ sudo systemctl enable wg-quick@wg0
Created symlink /etc/systemd/system/multi-user.target.wants/wg-quick@wg0.service → /usr/lib/systemd/system/wg-quick@.service.
ubuntu@ingress:~$ sudo wg show
interface: wg0
  public key: bpYKVMPlIRLagKzqetD0EDU3JQV5y5XuL0Vgc54/2V4=
  private key: (hidden)
  listening port: 54027

peer: SiigLgsvUml5tueNmiFy3eMSk5NcfHx2ba+anpOKi34=
  endpoint: 10.0.129.64:51820
  allowed ips: 10.8.0.1/32
  transfer: 0 B received, 296 B sent
  persistent keepalive: every 25 seconds
ubuntu@ingress:~$
```

Slika 24: Zagon WireGuard storitve na Ingress — `peer` kaže `Endpoint 10.0.129.64:51820`, `persistent keepalive 25 sekund`

10.1 Preizkus delovanja

Po vzpostavitvi tunela sem preizkusil povezavo z ukazom `ping 10.8.0.1` z Ingress-a. Trije uspešni odzivi z latenco okoli 0,7 ms potrjujejo, da paketi tečejo skozi šifriran tunel uspešno. Ukaz `wg show` poleg tega prikaže »latest handshake« in dejansko število prenesenih bajtov, kar je dokaz aktivne seje.

```
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data:
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=0.035 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=0.038 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=0.047 ms

--- 10.8.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2030ms
rtt min/avg/max/mdev = 0.035/0.040/0.047/0.005 ms
ubuntu@vault:/tmp$ ^C
ubuntu@vault:/tmp$ sudo wg show
interface: wg0
  public key: SiigLgsvUml5tueNmiFy3eMSk5NcfHx2ba+anpOKi34=
  private key: (hidden)
  listening port: 51820

peer: bpYKVMPlIRLagKzqetD0EDU3JQV5y5XuL0Vgc54/2V4=
  endpoint: 10.0.8.58:48648
  allowed ips: 10.8.0.2/32
  latest handshake: 1 minute, 25 seconds ago
  transfer: 660 B received, 476 B sent
ubuntu@vault:/tmp$
```

Slika 25: Tunel deluje (pogled iz Vault) — ping 10.8.0.1 brez paketnih izgub, »latest handshake« in transfer prikazujeta aktivno sejo

```
ubuntu@ingress:~$
ubuntu@ingress:~$ ping -c 3 10.8.0.1
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data.
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=0.698 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=0.641 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=0.851 ms

--- 10.8.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2062ms
rtt min/avg/max/mdev = 0.641/0.730/0.851/0.088 ms
ubuntu@ingress:~$ ^C
ubuntu@ingress:~$ sudo wg show
interface: wg0
  public key: bpYKVMP1IRLagKzqetD0EDU3JQV5y5XuL0Vgc54/2V4=
  private key: (hidden)
  listening port: 48648

peer: SiigLgsvUml5tueNmiFy3eMSk5NcfHx2bA+anpOKi34=
  endpoint: 10.0.129.64:51820
  allowed ips: 10.8.0.1/32
  latest handshake: 1 minute, 20 seconds ago
  transfer: 476 B received, 660 B sent
  persistent keepalive: every 25 seconds
ubuntu@ingress:~$
```

Slika 26: Tunel deluje (pogled iz Ingress) — ping 10.8.0.1 z latenco ~0,7 ms

11 Nextcloud – aplikacijski sloj

Nextcloud je odprtokodna platforma za samostojno gostovan oblak, primerljiva z Dropboxom ali Google Drive — z razliko, da podatki ostanejo v lastni infrastrukturi. Podpira sinhronizacijo datotek, deljenje in spletni vmesnik za upravljanje vsebine. V tem projektu sem ga postavil kot aplikacijski sloj na vstopnem vozlišču.

11.1 Priprava strukture

Najprej sem na Ingress-u ustvaril direktorijско strukturo za Docker Compose postavitev. Direktorij ~/nextcloud vsebuje data (Nextcloud uporabniške datoteke), db (MariaDB podatki), caddy_data in caddy_config (Caddy konfiguracija in TLS certifikati). Vsi ti direktoriji bodo vezani v posamezne vsebnike kot persistent volumes.

```
ubuntu@ingress:~$ ^C
ubuntu@ingress:~$ mkdir -p ~/nextcloud
ubuntu@ingress:~$ cd ~/nextcloud
ubuntu@ingress:~/nextcloud$ mkdir -p data db caddy_data caddy_config
ubuntu@ingress:~/nextcloud$ ls
caddy_config  caddy_data  data  db
ubuntu@ingress:~/nextcloud$
```

Slika 27: Priprava direktorijske strukture za Nextcloud Docker Compose postavitev

11.2 Docker Compose

Postavitev temelji na Docker Compose datoteki, ki definira tri storitve: db (MariaDB 11) za podatkovno bazo, app (Nextcloud 30 z Apache strežnikom) za samo aplikacijo in caddy

(Caddy 2) kot povratni proxy. Vsi vsebniki komunicirajo prek izoliranega bridge omrežja »ncnet«; samo Caddy izpostavlja vrata 80 in 443 navzven.

```

GNU nano 7.2 docker-compose.yml
services:
  db:
    image: mariadb:11
    container_name: nc-db
    restart: Unless-stopped
    command: --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-format=ROW
    volumes:
      - ./db:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: $(MYSQL_ROOT_PASSWORD)
      MYSQL_PASSWORD: $(MYSQL_PASSWORD)
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud
    networks: [ncnet]

  app:
    image: nextcloud:30-apache
    container_name: nc-app
    restart: Unless-stopped
    depends_on: [db]
    volumes:
      - ./data:/var/www/html
    environment:
      MYSQL_HOST: db
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud
      MYSQL_PASSWORD: $(MYSQL_PASSWORD)
      NEXTCLOUD_ADMIN_USER: $(NEXTCLOUD_ADMIN_USER)
      NEXTCLOUD_ADMIN_PASSWORD: $(NEXTCLOUD_ADMIN_PASSWORD)
      NEXTCLOUD_TRUSTED_DOMAINS: 63.183.210.104
      OVERWRITEPROTOCOL: https
      OVERWRITEHOST: YOUR.PUBLIC.IP.HERE
    networks: [ncnet]

  caddy:
    image: caddy:2-alpine
    container_name: nc-caddy
    restart: Unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./caddyfile:/etc/caddy/Caddyfile:ro
      - ./caddy_data:/data
      - ./caddy_config:/config
    networks: [ncnet]

networks:
  ncnet:
    driver: bridge

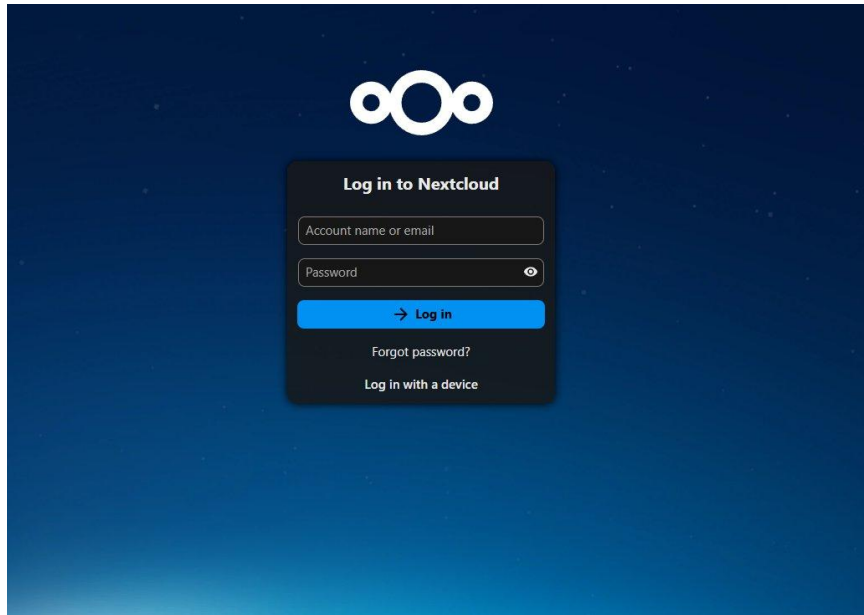
```

Slika 28: docker-compose.yml za Nextcloud — tri storitve povezane preko izoliranega ncnet omrežja, gesla iz .env datoteke

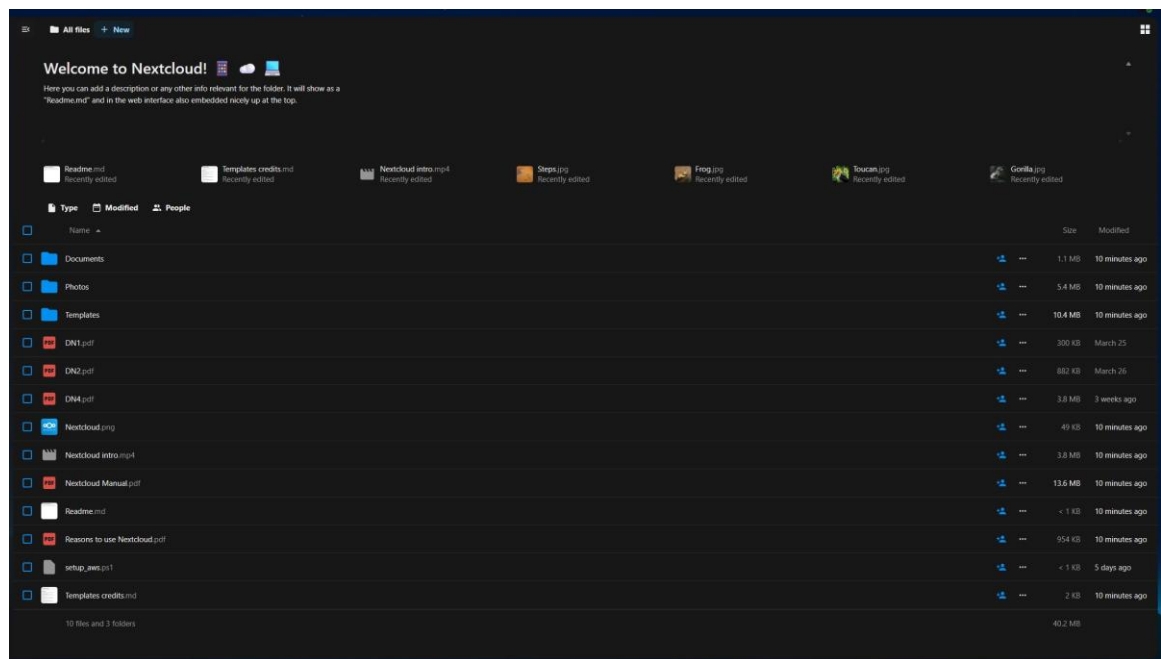
Caddy je bil izbran namesto bolj razširjenega nginx-a zaradi avtomatičnega upravljanja TLS certifikatov in bistveno preprostejše konfiguracije. V projektu Nextcloud teče preko HTTP, ker se uporablja gol IP naslov namesto domene — TLS certifikati za IP naslove imajo znane težave z združljivostjo. V produkcijski postavitvi bi bilo dovolj zamenjati IP s pravim domenskim imenom in Caddy bi avtomatično pridobil Let's Encrypt certifikat.

11.3 Spletni vmesnik in testne datoteke

Po zagonu vsebnikov je Nextcloud dosegljiv preko javnega IP naslova vstopnega vozlišča. Prijava z administratorskimi poverilnicami iz .env datoteke odpre uporabniški vmesnik. Za potrebe demonstracije sistema sem naložil več testnih datotek (PDF dokumente, slike, Markdown datoteke), ki bodo služile kot »uporabniški podatki« v testih varnostnega kopiranja.



Slika 29: Prijavna stran Nextcloud — dosegljiva preko javnega IP vstopnega vozlišča



Slika 30: Nadzorna plošča Nextcloud z naloženimi testnimi datotekami — 10 datotek in 3 mape, skupaj 40,2 MB

12 Restic REST strežnik na Vault

Restic je orodje za varnostno kopiranje. Podpira deduplikacijo (enaki bloki podatkov se shranijo samo enkrat), kompresijo in šifriranje. Strežniški del (rest-server) sem postavil na Vault v Docker vsebniku.

12.1 Prenos Docker slike na izolirano vozlišče

Ker Vault nima interneta, sem Docker sliko restic/rest-server:0.13.0 najprej prenesel na Ingress z docker pull, jo izvozil v tar arhiv z docker save (7,6 MB) in nato z scp prenesel na Vault preko zasebnega omrežja. Tam sem jo z docker load uvozil v lokalni Docker registry.

```
ubuntu@ingress:~/nextcloud$ docker pull restic/rest-server:0.13.0
0.13.0: Pulling from restic/rest-server
50b20dd743a: Pull complete
2838e3845b7: Pull complete
b44028b0656: Pull complete
bcc75b82687: Pull complete
088a9a1eebe: Pull complete
6a83fedfae6: Pull complete
Digest: sha256:8668c235a932745585fef5cabbab9eccb501035f90138085a6e74963dcdla71f
Status: Downloaded newer image for restic/rest-server:0.13.0
docker.io/restic/rest-server:0.13.0
ubuntu@ingress:~/nextcloud$ docker save restic/rest-server:0.13.0 -o /tmp/rest-server.tar
ubuntu@ingress:~/nextcloud$ ls -lh /tmp/rest-server.tar
-rw-r--r-- 1 ubuntu ubuntu 7.6M Apr 27 11:39 /tmp/rest-server.tar
ubuntu@ingress:~/nextcloud$ scp /tmp/rest-server.tar ubuntu@10.0.129.64:/tmp/
rest-server.tar
ubuntu@ingress:~/nextcloud$
```

Slika 31: Prenos Docker slike restic/rest-server:0.13.0 — pull na Ingress, save v tar arhiv, nato scp na Vault

12.2 HTTP osnovna avtentikacija

Restic REST strežnik podpira HTTP osnovno avtentikacijo z bcrypt-zgoščenimi gesli, shranjenimi v .htpasswd datoteki. Ker Vault nima orodij za bcrypt zgostitev, sem za enkratno generacijo uporabil Apache httpd Docker sliko, ki vsebuje htpasswd ukaz. Generirano zgoščeno geslo sem shranil v /home/ubuntu/restic/data/.htpasswd.

```
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
httpd:2.4-alpine          968c8b4098fc          97.9MB          22.1MB
ubuntu@vault:~/restic$ docker run --rm httpd:2.4-alpine htpasswd -nbB ingress 'VK0b9nYlKVmmRK' > .htpasswd
ubuntu@vault:~/restic$ cat .htpasswd
ingress:$2y$05$irXH0dvIqsplfGhINhBu50ZvAu6D6u3JwTnC4TkI2w1M9HsITv7ky

ubuntu@vault:~/restic$ chmod 600 .htpasswd
ubuntu@vault:~/restic$
```

Slika 32: Generiranje bcrypt zgoščene HTTP osnovne avtentikacije — uporabniško ime »ingress«, geslo zgoščeno z bcrypt

12.3 Docker Compose za Restic

Docker Compose datoteka za Restic REST strežnik definira eno storitev z dvema kritičnima varnostnima nastavitvama. Prva: vrata so vezana na specifični IP 10.8.0.1:8000 (WireGuard naslov), ne na 0.0.0.0. To pomeni, da storitev posluša izključno na tunelnem vmesniku in je drugje dejansko nedosegljiva. Druga: --no-auth=false v ukaznem parametru aktivira HTTP osnovno avtentikacijo z .htpasswd datoteke.

```

GNU nano 7.2 docker-compose.yml
services:
  rest:
    image: restic/rest-server:0.13.0
    container_name: restic-rest
    restart: unless-stopped
    environment:
      OPTIONS: "--no-auth --debug"
    command: ["--listen", ":8000", "--path", "/data", "--htpasswd-file", "/auth/.htpasswd", "--no-auth=false"]
    volumes:
      - ./data:/data
      - ../htpasswd:/auth/.htpasswd:ro
    ports:
      - "10.8.0.1:8000:8000"

```

Slika 33: docker-compose.yml za Restic REST na Vault — vrata vezana na WireGuard naslov 10.8.0.1:8000, ne na 0.0.0.0

12.4 Inicializacija Restic repozitorija

Z Ingress-a sem nato inicializiral Restic repozitorij. Restic za to potrebuje dva podatka: URL repozitorija (rest:http://ingress:GESLO@10.8.0.1:8000/) in šifrirno geslo (RESTIC_PASSWORD), ki ga uporablja za AES-256 šifriranje vsakega bloka podatkov. Pomembno: šifrirno geslo se nikoli ne pošlje na Vault — uporablja se izključno lokalno na Ingress-u.

```

ubuntu@ingress:~/nextcloud$ echo "Pass: $RESTIC_PASSWORD"
Pass: VK0b9nYlKVmmRK
ubuntu@ingress:~/nextcloud$ restic init
created restic repository b0b95f9d2c at rest:http://ingress:***@10.8.0.1:8000/

Please note that knowledge of your password is required to access
the repository. Losing your password means that your data is
irrecoverably lost.
ubuntu@ingress:~/nextcloud$ restic backup /etc/hostname
repository b0b95f9d opened (version 2, compression level auto)
created new cache in /home/ubuntu/.cache/restic
no parent snapshot found, will read all files

Files:          1 new,      0 changed,      0 unmodified
Dirs:           1 new,      0 changed,      0 unmodified
Added to the repository: 709 B (670 B stored)

processed 1 files, 8 B in 0:00
snapshot laf12ld8 saved
ubuntu@ingress:~/nextcloud$ restic snapshots
repository b0b95f9d opened (version 2, compression level auto)
ID            Time                Host      Tags      Paths
-----
laf12ld8     2026-04-27 11:51:35  ingress             /etc/hostname
1 snapshots
ubuntu@ingress:~/nextcloud$

```

Slika 34: Inicializacija Restic repozitorija z restic init — prva testna varnostna kopija /etc/hostname potrjuje delovanje celotne verige

Avtentikacija proti Vault-u poteka v treh neodvisnih nivojih (defense in depth). WireGuard zaklene povezavo na omrežnem nivoju: brez podpisa s pravilnim privatnim ključem se paket sploh ne prenaša. Restic REST nato zahteva HTTP osnovno avtentikacijo z bcrypt-zgoščenim geslom. Restic na koncu vsak blok podatkov šifrira z AES-256 in podpiše s Poly1305, preden ga pošlje iz vstopnega vozlišča.

13 Funkcija »preveri-in-uniči«

Mapa Staging znotraj Nextcloud-a je namenjena občutljivim datotekam, ki morajo biti čim prej premaknjene v izolirano arhivsko vozlišče in odstranjene z vstopnega vozlišča. To ustreza principu minimizacije podatkov (data minimization) iz zakonodaje GDPR. Skripta `/opt/backup/staging-cleanup.sh` izvaja naslednji življenjski cikel.

1. Pred kopiranjem zajame inventar map (imena datotek in velikosti).
2. Restic ustvari posnetek mape, skripta zajame ID posnetka.
3. Iz posnetka prebere seznam datotek z restic ls in primerja z lokalnim inventarjem. Ob neujemanju ABORT.
4. Izvede restic check --read-data, ki dešifrira 100 % blokov v posnetku in preveri njihovo celovitost.
5. Šele po obeh uspešnih preverjanjih izvede shred -n 3 -u za vsako datoteko (trikratna prepisava z naključnimi podatki, nato unlink).
6. Obvesti Nextcloud z ukazom `occ files:scan`, da osveži stanje v podatkovni bazi.

```
ubuntu@ingress: ~/nextcloud
GNU nano 7.2 /opt/backup/staging-cleanup.sh
#!/bin/bash
Verify-and-shred: backup the Staging folder, confirm integrity, then securely wipe it.
Implements data minimization: sensitive data exists on the public-facing ingress
only for the duration required to verify replication to the air-gapped Vault.
set -euo pipefail

source /root/.restic-env

STAGING="/home/ubuntu/nextcloud/data/data/admin/files/Staging"
LOG="/var/log/backup/staging-cleanup-$(date +%Ym%d-%H%M%S).log"
exec >> (tee -a "$LOG") 2>&1

echo "===== Staging cleanup started: $(date -Iseconds) ====="

--- Guard: nothing to do if staging is empty ---
if [ -z "$(ls -A "$STAGING" 2>/dev/null | grep -v '\.' || true)" ]; then
    echo "Staging directory empty. Nothing to back up. Exiting."
    exit 0
fi

1
Capture file inventory BEFORE backup, for verification
INVENTORY_BEFORE=$(find "$STAGING" -type f -printf '%P\ts\n' | sort)
FILE_COUNT=$(echo "$INVENTORY_BEFORE" | wc -l)
TOTAL_SIZE=$(find "$STAGING" -type f -printf '%s\n' | awk '{sum+=$1} END {print sum+0}')
echo ">>> Files in staging: $FILE_COUNT, total bytes: $TOTAL_SIZE"

--- 1. Run a tagged backup of staging only ---
echo ">>> Backing up staging directory..."
SNAPSHOT_ID=$(restic backup --tag staging-cleanup --json "$STAGING" \
    | tee -a "$LOG" \
    | grep -oP "snapshot_id":'[A-Za-z0-9_@-]+' | tail -1)

if [ -z "$SNAPSHOT_ID" ]; then
    echo "FATAL: Could not determine snapshot ID. Aborting cleanup."
    exit 1
fi
echo ">>> Snapshot created: $SNAPSHOT_ID"

--- 2. Verify the snapshot from Vault's side: list its contents and compare ---
echo ">>> Verifying snapshot contents against original..."
INVENTORY_FROM_BACKUP=$(restic ls "$SNAPSHOT_ID" --long \
    | awk '{S1=$3} {S2=$4} {S3=$5} {S4=$6} {S5=$7} {S6=$8} {S7=$9} {S8=$10} {S9=$11} {S10=$12} {S11=$13} {S12=$14} {S13=$15} {S14=$16} {S15=$17} {S16=$18} {S17=$19} {S18=$20} {S19=$21} {S20=$22} {S21=$23} {S22=$24} {S23=$25} {S24=$26} {S25=$27} {S26=$28} {S27=$29} {S28=$30} {S29=$31} {S30=$32} {S31=$33} {S32=$34} {S33=$35} {S34=$36} {S35=$37} {S36=$38} {S37=$39} {S38=$40} {S39=$41} {S40=$42} {S41=$43} {S42=$44} {S43=$45} {S44=$46} {S45=$47} {S46=$48} {S47=$49} {S48=$50} {S49=$51} {S50=$52} {S51=$53} {S52=$54} {S53=$55} {S54=$56} {S55=$57} {S56=$58} {S57=$59} {S58=$60} {S59=$61} {S60=$62} {S61=$63} {S62=$64} {S63=$65} {S64=$66} {S65=$67} {S66=$68} {S67=$69} {S68=$70} {S69=$71} {S70=$72} {S71=$73} {S72=$74} {S73=$75} {S74=$76} {S75=$77} {S76=$78} {S77=$79} {S78=$80} {S79=$81} {S80=$82} {S81=$83} {S82=$84} {S83=$85} {S84=$86} {S85=$87} {S86=$88} {S87=$89} {S88=$90} {S89=$91} {S90=$92} {S91=$93} {S92=$94} {S93=$95} {S94=$96} {S95=$97} {S96=$98} {S97=$99} {S98=$100} {S99=$101} {S100=$102} {S101=$103} {S102=$104} {S103=$105} {S104=$106} {S105=$107} {S106=$108} {S107=$109} {S108=$110} {S109=$111} {S110=$112} {S111=$113} {S112=$114} {S113=$115} {S114=$116} {S115=$117} {S116=$118} {S117=$119} {S118=$120} {S119=$121} {S120=$122} {S121=$123} {S122=$124} {S123=$125} {S124=$126} {S125=$127} {S126=$128} {S127=$129} {S128=$130} {S129=$131} {S130=$132} {S131=$133} {S132=$134} {S133=$135} {S134=$136} {S135=$137} {S136=$138} {S137=$139} {S138=$140} {S139=$141} {S140=$142} {S141=$143} {S142=$144} {S143=$145} {S144=$146} {S145=$147} {S146=$148} {S147=$149} {S148=$150} {S149=$151} {S150=$152} {S151=$153} {S152=$154} {S153=$155} {S154=$156} {S155=$157} {S156=$158} {S157=$159} {S158=$160} {S159=$161} {S160=$162} {S161=$163} {S162=$164} {S163=$165} {S164=$166} {S165=$167} {S166=$168} {S167=$169} {S168=$170} {S169=$171} {S170=$172} {S171=$173} {S172=$174} {S173=$175} {S174=$176} {S175=$177} {S176=$178} {S177=$179} {S178=$180} {S179=$181} {S180=$182} {S181=$183} {S182=$184} {S183=$185} {S184=$186} {S185=$187} {S186=$188} {S187=$189} {S188=$190} {S189=$191} {S190=$192} {S191=$193} {S192=$194} {S193=$195} {S194=$196} {S195=$197} {S196=$198} {S197=$199} {S198=$200} {S199=$201} {S200=$202} {S201=$203} {S202=$204} {S203=$205} {S204=$206} {S205=$207} {S206=$208} {S207=$209} {S208=$210} {S209=$211} {S210=$212} {S211=$213} {S212=$214} {S213=$215} {S214=$216} {S215=$217} {S216=$218} {S217=$219} {S218=$220} {S219=$221} {S220=$222} {S221=$223} {S222=$224} {S223=$225} {S224=$226} {S225=$227} {S226=$228} {S227=$229} {S228=$230} {S229=$231} {S230=$232} {S231=$233} {S232=$234} {S233=$235} {S234=$236} {S235=$237} {S236=$238} {S237=$239} {S238=$240} {S239=$241} {S240=$242} {S241=$243} {S242=$244} {S243=$245} {S244=$246} {S245=$247} {S246=$248} {S247=$249} {S248=$250} {S249=$251} {S250=$252} {S251=$253} {S252=$254} {S253=$255} {S254=$256} {S255=$257} {S256=$258} {S257=$259} {S258=$260} {S259=$261} {S260=$262} {S261=$263} {S262=$264} {S263=$265} {S264=$266} {S265=$267} {S266=$268} {S267=$269} {S268=$270} {S269=$271} {S270=$272} {S271=$273} {S272=$274} {S273=$275} {S274=$276} {S275=$277} {S276=$278} {S277=$279} {S278=$280} {S279=$281} {S280=$282} {S281=$283} {S282=$284} {S283=$285} {S284=$286} {S285=$287} {S286=$288} {S287=$289} {S288=$290} {S289=$291} {S290=$292} {S291=$293} {S292=$294} {S293=$295} {S294=$296} {S295=$297} {S296=$298} {S297=$299} {S298=$300} {S299=$301} {S300=$302} {S301=$303} {S302=$304} {S303=$305} {S304=$306} {S305=$307} {S306=$308} {S307=$309} {S308=$310} {S309=$311} {S310=$312} {S311=$313} {S312=$314} {S313=$315} {S314=$316} {S315=$317} {S316=$318} {S317=$319} {S318=$320} {S319=$321} {S320=$322} {S321=$323} {S322=$324} {S323=$325} {S324=$326} {S325=$327} {S326=$328} {S327=$329} {S328=$330} {S329=$331} {S330=$332} {S331=$333} {S332=$334} {S333=$335} {S334=$336} {S335=$337} {S336=$338} {S337=$339} {S338=$340} {S339=$341} {S340=$342} {S341=$343} {S342=$344} {S343=$345} {S344=$346} {S345=$347} {S346=$348} {S347=$349} {S348=$350} {S349=$351} {S350=$352} {S351=$353} {S352=$354} {S353=$355} {S354=$356} {S355=$357} {S356=$358} {S357=$359} {S358=$360} {S359=$361} {S360=$362} {S361=$363} {S362=$364} {S363=$365} {S364=$366} {S365=$367} {S366=$368} {S367=$369} {S368=$370} {S369=$371} {S370=$372} {S371=$373} {S372=$374} {S373=$375} {S374=$376} {S375=$377} {S376=$378} {S377=$379} {S378=$380} {S379=$381} {S380=$382} {S381=$383} {S382=$384} {S383=$385} {S384=$386} {S385=$387} {S386=$388} {S387=$389} {S388=$390} {S389=$391} {S390=$392} {S391=$393} {S392=$394} {S393=$395} {S394=$396} {S395=$397} {S396=$398} {S397=$399} {S398=$400} {S399=$401} {S400=$402} {S401=$403} {S402=$404} {S403=$405} {S404=$406} {S405=$407} {S406=$408} {S407=$409} {S408=$410} {S409=$411} {S410=$412} {S411=$413} {S412=$414} {S413=$415} {S414=$416} {S415=$417} {S416=$418} {S417=$419} {S418=$420} {S419=$421} {S420=$422} {S421=$423} {S422=$424} {S423=$425} {S424=$426} {S425=$427} {S426=$428} {S427=$429} {S428=$430} {S429=$431} {S430=$432} {S431=$433} {S432=$434} {S433=$435} {S434=$436} {S435=$437} {S436=$438} {S437=$439} {S438=$440} {S439=$441} {S440=$442} {S441=$443} {S442=$444} {S443=$445} {S444=$446} {S445=$447} {S446=$448} {S447=$449} {S448=$450} {S449=$451} {S450=$452} {S451=$453} {S452=$454} {S453=$455} {S454=$456} {S455=$457} {S456=$458} {S457=$459} {S458=$460} {S459=$461} {S460=$462} {S461=$463} {S462=$464} {S463=$465} {S464=$466} {S465=$467} {S466=$468} {S467=$469} {S468=$470} {S469=$471} {S470=$472} {S471=$473} {S472=$474} {S473=$475} {S474=$476} {S475=$477} {S476=$478} {S477=$479} {S478=$480} {S479=$481} {S480=$482} {S481=$483} {S482=$484} {S483=$485} {S484=$486} {S485=$487} {S486=$488} {S487=$489} {S488=$490} {S489=$491} {S490=$492} {S491=$493} {S492=$494} {S493=$495} {S494=$496} {S495=$497} {S496=$498} {S497=$499} {S498=$500} {S499=$501} {S500=$502} {S501=$503} {S502=$504} {S503=$505} {S504=$506} {S505=$507} {S506=$508} {S507=$509} {S508=$510} {S509=$511} {S510=$512} {S511=$513} {S512=$514} {S513=$515} {S514=$516} {S515=$517} {S516=$518} {S517=$519} {S518=$520} {S519=$521} {S520=$522} {S521=$523} {S522=$524} {S523=$525} {S524=$526} {S525=$527} {S526=$528} {S527=$529} {S528=$530} {S529=$531} {S530=$532} {S531=$533} {S532=$534} {S533=$535} {S534=$536} {S535=$537} {S536=$538} {S537=$539} {S538=$540} {S539=$541} {S540=$542} {S541=$543} {S542=$544} {S543=$545} {S544=$546} {S545=$547} {S546=$548} {S547=$549} {S548=$550} {S549=$551} {S550=$552} {S551=$553} {S552=$554} {S553=$555} {S554=$556} {S555=$557} {S556=$558} {S557=$559} {S558=$560} {S559=$561} {S560=$562} {S561=$563} {S562=$564} {S563=$565} {S564=$566} {S565=$567} {S566=$568} {S567=$569} {S568=$570} {S569=$571} {S570=$572} {S571=$573} {S572=$574} {S573=$575} {S574=$576} {S575=$577} {S576=$578} {S577=$579} {S578=$580} {S579=$581} {S580=$582} {S581=$583} {S582=$584} {S583=$585} {S584=$586} {S585=$587} {S586=$588} {S587=$589} {S588=$590} {S589=$591} {S590=$592} {S591=$593} {S592=$594} {S593=$595} {S594=$596} {S595=$597} {S596=$598} {S597=$599} {S598=$600} {S599=$601} {S600=$602} {S601=$603} {S602=$604} {S603=$605} {S604=$606} {S605=$607} {S606=$608} {S607=$609} {S608=$610} {S609=$611} {S610=$612} {S611=$613} {S612=$614} {S613=$615} {S614=$616} {S615=$617} {S616=$618} {S617=$619} {S618=$620} {S619=$621} {S620=$622} {S621=$623} {S622=$624} {S623=$625} {S624=$626} {S625=$627} {S626=$628} {S627=$629} {S628=$630} {S629=$631} {S630=$632} {S631=$633} {S632=$634} {S633=$635} {S634=$636} {S635=$637} {S636=$638} {S637=$639} {S638=$640} {S639=$641} {S640=$642} {S641=$643} {S642=$644} {S643=$645} {S644=$646} {S645=$647} {S646=$648} {S647=$649} {S648=$650} {S649=$651} {S650=$652} {S651=$653} {S652=$654} {S653=$655} {S654=$656} {S655=$657} {S656=$658} {S657=$659} {S658=$660} {S659=$661} {S660=$662} {S661=$663} {S662=$664} {S663=$665} {S664=$666} {S665=$667} {S666=$668} {S667=$669} {S668=$670} {S669=$671} {S670=$672} {S671=$673} {S672=$674} {S673=$675} {S674=$676} {S675=$677} {S676=$678} {S677=$679} {S678=$680} {S679=$681} {S680=$682} {S681=$683} {S682=$684} {S683=$685} {S684=$686} {S685=$687} {S686=$688} {S687=$689} {S688=$690} {S689=$691} {S690=$692} {S691=$693} {S692=$694} {S693=$695} {S694=$696} {S695=$697} {S696=$698} {S697=$699} {S698=$700} {S699=$701} {S700=$702} {S701=$703} {S702=$704} {S703=$705} {S704=$706} {S705=$707} {S706=$708} {S707=$709} {S708=$710} {S709=$711} {S710=$712} {S711=$713} {S712=$714} {S713=$715} {S714=$716} {S715=$717} {S716=$718} {S717=$719} {S718=$720} {S719=$721} {S720=$722} {S721=$723} {S722=$724} {S723=$725} {S724=$726} {S725=$727} {S726=$728} {S727=$729} {S728=$730} {S729=$731} {S730=$732} {S731=$733} {S732=$734} {S733=$735} {S734=$736} {S735=$737} {S736=$738} {S737=$739} {S738=$740} {S739=$741} {S740=$742} {S741=$743} {S742=$744} {S743=$745} {S744=$746} {S745=$747} {S746=$748} {S747=$749} {S748=$750} {S749=$751} {S750=$752} {S751=$753} {S752=$754} {S753=$755} {S754=$756} {S755=$757} {S756=$758} {S757=$759} {S758=$760} {S759=$761} {S760=$762} {S761=$763} {S762=$764} {S763=$765} {S764=$766} {S765=$767} {S766=$768} {S767=$769} {S768=$770} {S769=$771} {S770=$772} {S771=$773} {S772=$774} {S773=$775} {S774=$776} {S775=$777} {S776=$778} {S777=$779} {S778=$780} {S779=$781} {S780=$782} {S781=$783} {S782=$784} {S783=$785} {S784=$786} {S785=$787} {S786=$788} {S787=$789} {S788=$790} {S789=$791} {S790=$792} {S791=$793} {S792=$794} {S793=$795} {S794=$796} {S795=$797} {S796=$798} {S797=$799} {S798=$800} {S799=$801} {S800=$802} {S801=$803} {S802=$804} {S803=$805} {S804=$806} {S805=$807} {S806=$808} {S807=$809} {S808=$810} {S809=$811} {S810=$812} {S811=$813} {S812=$814} {S813=$815} {S814=$816} {S815=$817} {S816=$818} {S817=$819} {S818=$820} {S819=$821} {S820=$822} {S821=$823} {S822=$824} {S823=$825} {S824=$826} {S825=$827} {S826=$828} {S827=$829} {S828=$830} {S829=$831} {S830=$832} {S831=$833} {S832=$834} {S833=$835} {S834=$836} {S835=$837} {S836=$838} {S837=$839} {S838=$840} {S839=$841} {S840=$842} {S841=$843} {S842=$844} {S843=$845} {S844=$846} {S845=$847} {S846=$848} {S847=$849} {S848=$850} {S849=$851} {S850=$852} {S851=$853} {S852=$854} {S853=$855} {S854=$856} {S855=$857} {S856=$858} {S857=$859} {S858=$860} {S859=$861} {S860=$862} {S861=$863} {S862=$864} {S863=$865} {S864=$866} {S865=$867} {S866=$868} {S867=$869} {S868=$870} {S869=$871} {S870=$872} {S871=$873} {S872=$874} {S873=$875} {S874=$876} {S875=$877} {S876=$878} {S877=$879} {S878=$880} {S879=$881} {S880=$882} {S881=$883} {S882=$884} {S883=$885} {S884=$886} {S885=$887} {S886=$888} {S887=$889} {S888=$890} {S889=$891} {S890=$892} {S891=$893} {S892=$894} {S893=$895} {S894=$896} {S895=$897} {S896=$898} {S897=$899} {S898=$900} {S899=$901} {S900=$902} {S901=$903} {S902=$904} {S903=$905} {S904=$906} {S905=$907} {S906=$908} {S907=$909} {S908=$910} {S909=$911} {S910=$912} {S911=$913} {S912=$914} {S913=$915} {S914=$916} {S915=$917} {S916=$918} {S917=$919} {S918=$920} {S919=$921} {S920=$922} {S921=$923} {S922=$924} {S923=$925} {S924=$926} {S925=$927} {S926=$928} {S927=$929} {S928=$930} {S929=$931} {S930=$932} {S931=$933} {S932=$934} {S933=$935} {S934=$936} {S935=$937} {S936=$938} {S937=$939} {S938=$940} {S939=$941} {S940=$942} {S941=$943} {S942=$944} {S943=$945} {S944=$946} {S945=$947} {S946=$948} {S947=$949} {S948=$950} {S949=$951} {S950=$952} {S951=$953} {S952=$954} {S953=$955} {S954=$956} {S955=$957} {S956=$958} {S957=$959} {S958=$960} {S959=$961} {S960=$962} {S961=$963} {S962=$964} {S963=$965} {S964=$966} {S965=$967} {S966=$968} {S967=$969} {S968=$970} {S969=$971} {S970=$972} {S971=$973} {S972=$974} {S973=$975} {S974=$976} {S975=$977} {S976=$978} {S977=$979} {S978=$980} {S979=$981} {S980=$982} {S981=$983} {S982=$984} {S983=$985} {S984=$986} {S985=$987} {S986=$988} {S987=$989} {S988=$990} {S989=$991} {S990=$992} {S991=$993} {S992=$994} {S993=$995} {S994=$996} {S995=$997} {S996=$998} {S997=$999} {S998=$1000} {S999=$1001} {S1000=$1002} {S1001=$1003} {S1002=$1004} {S1003=$1005} {S1004=$1006} {S1005=$1007} {S1006=$1008} {S1007=$1009} {S1008=$1010} {S1009=$1011} {S1010=$1012} {S1011=$1013} {S1012=$1014} {S1013=$1015} {S1014=$1016} {S1015=$1017} {S1016=$1018} {S1017=$1019} {S1018=$1020} {S1019=$1021} {S1020=$1022} {S1021=$1023} {S1022=$1024} {S1023=$1025} {S1024=$1026} {S1025=$1027} {S1026=$1028} {S1027=$1029} {S1028=$1030} {S1029=$1031} {S1030=$1032} {S1031=$1033} {S1032=$1034} {S1033=$1035} {S1034=$1036} {S1035=$1037} {S1036=$1038} {S1037=$1039} {S1038=$1040} {S1039=$1041} {S1040=$1042} {S1041=$1043} {S1042=$1044} {S1043=$1045} {S1044=$1046} {S1045=$1047} {S1046=$1048} {S1047=$1049} {S1048=$1050} {S1049=$1051} {S1050=$1052} {S1051=$1053} {S1052=$1054} {S1053=$1055} {S1054=$1056} {S1055=$1057} {S1056=$1058} {S1057=$1059} {S1058=$1060} {S1059=$1061} {S1060=$1062} {S1061=$1063} {S1062=$1064} {S1063=$1065} {S1064=$1066} {S1065=$1067} {S1066=$1068} {S1067=$1069} {S1068=$1070} {S1069=$1071} {S1070=$1072} {S1071=$1073} {S1072=$1074} {S1073=$1075} {S1074=$1076} {S1075=$1077} {S1076=$1078} {S1077=$1079} {S1078=$1080} {S1079=$1081} {S1080=$1082} {S1081=$1083} {S1082=$1084} {S1083=$1085} {S1084=$1086} {S1085=$1087} {S1086=$1088} {S1087=$1089} {S1088=$1090} {S1089=$1091} {S1090=$1092} {S1091=$1093} {S1092=$1094} {S1093=$1095} {S1094=$1096} {S1095=$1097} {S1096=$1098} {S1097=$1099} {S1098=$1100} {S1099=$1101} {S1100=$1102} {S1101=$1103} {S1102=$1104} {S1103=$1105} {S1104=$1106} {S1105=$1107} {S1106=$1108} {S1107=$1109} {S1108=$1110} {S1109=$1111} {S1110=$1112} {S1111=$1113} {S1112=$1114} {S1113=$1115} {S1114=$1116} {S1115=$1117} {S1116=$1118} {S1117=$1119} {S1118=$1120} {S1119=$1121} {S1120=$1122} {S1121=$1123} {S1122=$1124} {S1123=$1125} {S1124=$1126} {S1125=$1127} {S1126=$1128} {S1127=$11
```

Slika 35: Skripta /opt/backup/staging-cleanup.sh — implementacija »preveri-in-uniči« vzorca

Med testiranjem je skripta brisanje pravilno prekinila zaradi napake v parsanju izhoda ukaza `restic ls` (napačen stolpec za velikost datoteke). Ker invariant »preverjanje mora uspeti pred brisanjem« v skripti drži, ni bila izbrisana nobena datoteka. Tak izid je pravzaprav ravno smisel zasnove: če sistem ne ve zagotovo, da je kopija dobra, raje pusti original pri miru.

14 Test obnove podatkov (DR test)

Pri sysadminih velja pravilo: kopija, ki ni bila nikoli obnovljena, ne šteje za kopijo. Da bi preveril, da sistem dejansko deluje, sem izvedel popoln test obnove podatkov. Postopek simulira ransomware napad: izberem ciljno datoteko, izračunam njeno SHA-256 zgoščeno vrednost, datoteko izbrišem in jo nato obnovim iz Vault-a.

```
ubuntu@ingress:~/nextcloud$ # Capture pre-disaster baseline
ORIGINAL_HASH=$(sudo sha256sum "$TARGET" | awk '{print $1}')
echo "Original SHA-256: $ORIGINAL_HASH"
echo "$ORIGINAL_HASH $TARGET" | sudo tee /root/dr-test-baseline.txt
Original SHA-256: 83c21981976b200ad259f706d5943ee7452a2eab9875ba6f4642492b6287a3cc
83c21981976b200ad259f706d5943ee7452a2eab9875ba6f4642492b6287a3cc /home/ubuntu/nextcloud/data/data/admin/files/Documents/Readme.md
ubuntu@ingress:~/nextcloud$
```

Slika 36: Zajem SHA-256 referenčne vrednosti pred simulirano katastrofo — 83c21981...287a3cc shranjeno v /root/dr-test-baseline.txt

```
restoring <Snapshot 96702a84 of [/home/ubuntu/nextcloud/data/data /tmp/tmp.tsG7EVRZKO/nextcloud-db.sql.gz] at 2026-04-27 11:58:41.742082689 +0000 UTC by z
@ingress> to /tmp/dr-restore
Summary: Restored 169 files/dirs (41.576 MiB) in 0:00
ubuntu@ingress:~/nextcloud$ TARGET="/home/ubuntu/nextcloud/data/data/admin/files/Documents/Readme.md"
RESTORED="/tmp/dr-restore/$TARGET"

sudo ls -lh "$RESTORED"
RESTORED_HASH=$(sudo sha256sum "$RESTORED" | awk '{print $1}')
echo "Original: $ORIGINAL_HASH"
echo "Restored: $RESTORED_HASH"
[ "$ORIGINAL_HASH" = "$RESTORED_HASH" ] && echo "DR TEST PASSED" || echo "DR TEST FAILED"
-rw-r--r-- 1 www-data www-data 136 Apr 27 11:26 /tmp/dr-restore/home/ubuntu/nextcloud/data/data/admin/files/Documents/Readme.md
Original: 83c21981976b200ad259f706d5943ee7452a2eab9875ba6f4642492b6287a3cc
Restored: 83c21981976b200ad259f706d5943ee7452a2eab9875ba6f4642492b6287a3cc
DR TEST PASSED
ubuntu@ingress:~/nextcloud$
```

Slika 37: Uspešen test obnove — obnova 169 datotek (41,6 MiB), primerjava SHA-256 zgoščenih vrednosti pred in po obnovi se ujema → DR TEST PASSED

Identični zgoščeni vrednosti SHA-256 dokazujeta, da je datoteka potovala skozi celotno verigo (Ingress → Restic šifriranje z AES-256 → HTTP avtentikacija → WireGuard tunel s ChaCha20-Poly1305 → Vault → tunel nazaj → dešifriranje → obnova) in se vrnila bit-natančno enaka. Skupaj je bilo obnovljenih 169 datotek velikosti 41,6 MiB v štirih sekundah.

Verjetnost slučajnega trka SHA-256 je 2 na minus 256, kar je za vse praktične namene zanemarljivo. Ujemanje torej ni statistično, ampak kriptografsko: dveh različnih datotek, ki bi imeli enako SHA-256 zgoščeno vrednost, v svetu nihče ne pozna.

15 Zaključek

15.1 Povzetek doseženega

V projektu sem zasnoval in postavil sistem za zaščito podatkov v AWS, ki temelji na izolaciji, šifriranju med prenosom in v mirovanju ter kriptografski avtentikaciji. Bistvo

arhitekture je ločitev javnega vstopnega vozlišča od izoliranega arhivskega vozlišča: napadalec, ki obvlada vstopni del, do arhiva ne pride.

Sistem sem preizkusil s popolno obnovo podatkov: SHA-256 zgoščene vrednosti pred izbrisom in po obnovi se ujemajo, kar pomeni bit-natančno celovitost. Skripta »preveri-in-uniči« je med testiranjem tudi sama prekinila brisanje zaradi napake pri parsanju Restic izhoda. V tem kontekstu je to dober rezultat: skripta je raje pustila datoteke pri miru, kot pa tvegala izgubo.

15.2 Znane omejitve

- Nextcloud teče preko HTTP namesto HTTPS zaradi težav s TLS certifikati za gole IP naslove. Produkcijska različica bi uporabila registrirano domeno z avtomatskim Let's Encrypt certifikatom.
- Šifrirno geslo Restic je shranjeno v plain-text datoteki /root/.restic-env. V produkciji bi bilo to v AWS Secrets Manager.
- Sistem ne pošilja samodejnih obvestil ob napaki kopiranja. V produkciji bi cron skripta poslala alarm preko CloudWatch.
- Trenutna postavitev ima samo eno kopijo v eni AWS regiji. Pravo izvajanje pravila 3-2-1 bi vsebovalo dodatno geo-replikacijo.

15.3 Možnosti za nadaljnji razvoj

- Implementacija Terraform ali CloudFormation predloge za infrastrukturo kot kodo.
- Dodajanje CloudWatch alarmov in samodejnih obvestil ob neuspešnih kopijah.
- Replikacija arhiva v drugo AWS regijo za pravo geo-redundanco.
- Migracija šifrirnega gesla Restic v AWS Secrets Manager.
- Dodajanje spletnega vmesnika Duplicati za vizualno upravljanje opravil kopiranja.

16 Viri in literature

- Amazon Web Services. EC2 User Guide. Dostopno na: <https://docs.aws.amazon.com/ec2/>
- Amazon Web Services. VPC Documentation. Dostopno na: <https://docs.aws.amazon.com/vpc/>
- Donenfeld, J. (2017). WireGuard: Next Generation Kernel Network Tunnel.
- Restic Project. Restic Documentation. Dostopno na: <https://restic.readthedocs.io/>
- Nextcloud GmbH. Nextcloud Server Documentation. Dostopno na: <https://docs.nextcloud.com/>
- Caddy Web Server. Documentation. Dostopno na: <https://caddyserver.com/docs/>
- Veeam Software. The 3-2-1 Backup Rule Explained.
- OWASP Foundation. Cryptographic Storage Cheat Sheet.
- Docker Inc. Docker Compose Documentation. Dostopno na: <https://docs.docker.com/compose/>

Nejc Dolinar ✓